



Guide étudiant (version 4)

GRO321 – APP4 Session S3

Programmation orientée objet et bases de
données relationnelles

Génie robotique
Faculté de génie
Université de Sherbrooke

Été 2026

Document GRO321-E26-Guide etudiant.docx

Rédigé par François Ferland et Eugène Morin (2026). Certains éléments adaptés des cours GEN241 (Eugène Morin et coll.) et GIF325 (Bernard Beaulieu et Domingo Palao)

Copyright © 2026, Faculté de génie, Université de Sherbrooke

Table des matières

1.	Activités pédagogiques et compétences.....	1
2.	Synthèse de l'évaluation	1
3.	Qualités de l'ingénieur	2
4.	Énoncé de la problématique.....	3
5.	Connaissances nouvelles.....	5
6.	Guide de lecture	6
6.1.	Séquence d'étude suggérée	6
7.	Environnement logiciel.....	6
7.1.	Consigne sur l'utilisation de l'IA générative	6
8.	Productions à remettre	7
8.1.	Formation des équipes	7
8.2.	Contraintes et pénalités pour les livrables	7
8.3.	Livrables	7
9.	Évaluations.....	9
9.1.	Rapport d'APP et validation	9
9.2.	Évaluation sommative.....	9
9.3.	Évaluation finale	9
9.4.	Grille d'évaluation du rapport et de la validation	10
10.	Activités procédurale 1	11
10.1.	Programmation orientée objet.....	11
10.2.	Bases de données	16
11.	Activité en laboratoire.....	19
11.1.	Préparation	19
11.2.	Exercice 1 - Des classes pour les formes	19
11.3.	Exercice 2 - SQL en ligne de commande.....	20
11.4.	Exercice 3 - Des objets à la base.....	21
11.5.	Pour la suite.....	21
12.	Activités procédurale 2	22
12.1.	Programmation orientée objet : Classes abstraites et surcharge de fonctions	22
12.2.	Diagrammes UML	23
12.3.	Base de données : Normalisation.....	25
13.	Validation pratique de la solution à la problématique	28
14.	Deuxième tutorat.....	28

1. Activités pédagogiques et compétences

GRO321 : Programmation orientée objet et bases de données relationnelles

1. Modéliser et mettre en œuvre un logiciel selon le paradigme de la programmation orientée objet selon les meilleures pratiques de documentation et de validation de l'implémentation obtenue.
2. Résoudre un problème de gestion de l'information par la modélisation et la mise en œuvre d'une base de données relationnelle.

2. Synthèse de l'évaluation

La note attribuée aux activités pédagogiques de l'APP est une note individuelle, sauf pour le rapport d'APP qui est une note par équipe de deux. L'évaluation porte sur les compétences figurant dans la description des activités pédagogiques de l'APP à la section 1. Ces compétences, ainsi que la pondération (sur un total de 4500 points pour une session de 15 crédits) de chacune d'entre elles dans l'évaluation de l'unité, sont :

<i>Activité pédagogique</i>	GRO321	
<i>Compétence</i>	C1 Prog. orientée objet	C2 Bases de données
<i>Livrables</i>		
Rapport d'APP	30	30
Validation	15	15
Évaluation sommative	135	135
Évaluation finale	120	120
Total	300	300

3. Qualités de l'ingénieur

Les qualités de l'ingénieur visées par cette unité d'APP sont les suivantes. D'autres qualités peuvent être présentes sans être visées ou évaluées dans cette unité d'APP.

	Q01	Q02	Q03	Q04	Q05	Q06	Q07	Q08	Q09	Q10	Q11	Q12
Touchée	x	x	x		x							
Évaluée	x	x	x		x							

Les qualités de l'ingénieur sont les suivantes. Pour une description détaillée des qualités et leur provenance, consultez le lien suivant :

<https://www.usherbrooke.ca/genie/futurs-etudiants/programmes-de-1er-cycle/baccalaureats/bcapg>

Qualité	Libellé
Q01	Connaissances en génie
Q02	Analyse de problèmes
Q03	Investigation
Q04	Conception
Q05	Utilisation d'outils d'ingénierie
Q06	Travail individuel et en équipe
Q07	Communication
Q08	Professionnalisme
Q09	Impact du génie sur la société et l'environnement
Q10	Déontologie et équité
Q11	Économie et gestion de projets
Q12	Apprentissage continu

4. Énoncé de la problématique

Gestion efficace d'une flotte de robots de service

La robotique de service telle que les plateformes mobiles employées dans les restaurants et hôtels pour livrer des articles à la clientèle est en pleine croissance et permet notamment d'alléger la charge de travail du personnel de service et ainsi leur permettre de se concentrer à des tâches à plus grande valeur ajoutée. Dans la grande majorité des situations, ces robots ne sont pas opérés par les établissements eux-mêmes, mais plutôt par des entreprises spécialisées dans l'intégration, la gestion et l'entretien de tels robots. On appelle parfois ce modèle d'affaire *Robot As A Service* (RaaS), qui est calqué sur celui déjà bien présent dans les services de l'information ou des imprimantes-photocopieurs.

Vous venez de décrocher un emploi dans une telle entreprise de robots de service. La compagnie est en pleine croissance, ayant tout récemment obtenu un contrat avec un grand groupe de restaurateurs qui souhaite implanter des robots de services dans tous ses établissements de la province. La signature de ce contrat est une immense réussite pour l'entreprise en démarrage, mais elle est accompagnée d'un nombre considérable de défis. Un de ceux-ci concerne la maintenance des robots déployés un peu partout au Québec. En effet, si jusqu'à maintenant l'entreprise se débrouillait bien avec une dizaine de systèmes pratiquement identiques dans quelques établissements dans la région et pouvait se permettre d'envoyer ses ingénieurs s'occuper des tâches d'entretien ou de mise à jour du logiciel, elle est consciente que cette façon de faire ne tiendra pas la route avec plus d'une centaine de routes déployés aux quatre coins de la province.

C'est pour relever ce défi que vous avez été recruté. Vous avez développé une certaine expertise en implantation de procédés et protocoles de maintenance pour de l'équipement industriel, et votre mandat est de préparer la future équipe de techniciens et techniciennes mobiles pour une gestion efficace des robots de vos clients. Votre première tâche est d'implanter un système de Gestion de Maintenance Assistée par Ordinateur (GMAO, ou *Computerized Maintenance Management System* – CMMS). Vous êtes familier avec des systèmes populaires comme SAP¹, mais votre employeur n'a tout simplement pas les ressources pour payer les frais des consultants spécialisés dans ces produits.

Par chance, l'entreprise a déjà mis en place une base de données relationnelle pour suivre l'inventaire des robots et de ses clients. L'ensemble de la base de données a été défini et est exploité à l'aide du langage SQL. Après un survol de l'implémentation actuelle, vous constatez que plusieurs améliorations pourraient y être apportées. Par exemple, la table représentant l'ensemble des données clients et leurs robots n'est pas normalisée. Décomposer cette table de façon à respecter la troisième forme normale (3FN) serait déjà un gain appréciable. Cela dit, le système déjà en place et son interface web rudimentaire constituent un bon point de départ.

Pour les deux prochaines semaines, vous décidez de vous concentrer sur l'ajout d'une fonctionnalité pour les personnes chargées de la maintenance : la génération automatique de bons de travail. Ceux-ci permettront au technicien ou à la technicienne dépêchée chez le client de

¹ <https://www.sap.com/sea/resources/what-is-cmms>

connaître rapidement sur quel robot la maintenance doit être effectuée, en plus des tâches précises à effectuer, comme la vérification du système d'alimentation ou la mise à jour du logiciel de contrôle de la base mobile. Ces bons de travail devront être générée à même l'interface web déjà en place. Vous n'êtes pas familier avec ce type de développement, mais par chance un de vos collègues a déjà mis en place les éléments nécessaires. Il vous suffira de bien les intégrer avec le reste de vos changements.

En analysant le logiciel Python gérant l'interface, vous réalisez qu'elle a été structurée par objets. En effet, plusieurs classes représentent des concepts complets du système de réservation, comme les robots affectés ou les clients. Les fonctions répondant aux commandes des usagers à travers l'interface manipulent abondamment ces objets, en appelant des méthodes spécifiques pour, par exemple, modifier le numéro de contact d'un client ou créer des objets représentant de nouvelles entrées dans l'inventaire. Vous jugez donc qu'il sera important de respecter cette contrainte et représenter les bons de travail comme des objets à part entière. Comme il existe trois catégories de bons de travail, soient *Diagnostic*, *Mise à jour* et *Réparation*, il semble approprié de regrouper les éléments communs aux bons dans une classe parente et confier les fonctionnalités spécifiques à des classes dérivées de celles-ci.

En analysant davantage le prototype de travail de votre collègue, vous remarquez qu'il a laissé plusieurs points d'accès de l'interface comme "squelette" de code qui pourront être complétés. Plusieurs indications dans le code viennent clarifier le protocole de chaque fonction appelée par l'interface, ce qui devrait vous guider dans la résolution de la problématique. Votre collègue a également laissé une preuve de concept pour le modèle de données des bons de travail, mais il n'est pas arrimé au reste de la base de données, ni sous une forme normalisée. Il ne faudra pas négliger cet aspect.

Finalement, l'entreprise a mis en place plusieurs normes de documentation de ses logiciels que vous devrez respecter. Elle s'attend notamment à un diagramme de classes détaillé de toutes les entités de l'application, ce qui semble être absent pour l'instant. Un ensemble de diagrammes UML de séquences, états-transition et cas d'utilisation sont également attendus pour décrire le fonctionnement de la nouvelle fonctionnalité des bons de travail, en incluant les appels sous-jacents à la nouvelle fonctionnalité (si applicable).

5. Connaissances nouvelles

Connaissances déclaratives : *Quoi*

- Programmation orientée objet (GRO321-1) :
 - Notation UML (*Universal Modeling Language*) : Diagrammes de classes, séquences, états-transition et cas d'utilisation
 - Définition de classes: méthodes et propriétés
 - Héritage
 - Polymorphisme et surcharge de fonctions
 - Classes abstraites
- Bases de données (GRO321-2) :
 - Modèle conceptuel : Entités, attributs, liens, identifiants
 - Modèle relationnel : Relations, domaine, dépendance fonctionnelle et algèbre relationnelle
 - Définition des données : Tables, colonnes enregistrements et contraintes
 - Manipulation des données : Insertion, recherche, modifications, retraits.

Connaissances procédurales : *Comment*

- Modéliser une application orientée-objet et la documenter en utilisant la notation UML
- Modéliser des bases de données relationnelles
- Exploiter une base de données relationnelle à l'aide du langage SQL (*Structured Query Language*)

Connaissances conditionnelles : *Quand*

- Reconnaître lorsqu'une base de données relationnelles normalisée est nécessaire et bénéfique
- Regrouper des méthodes dans une classe parente pour mutualiser certaines fonctionnalités d'un ensemble d'entités

6. Guide de lecture

Ressources obligatoires :

- Swinnen, Gérard. « Apprendre à programmer avec Python 3 ». Disponible librement en ligne : <https://inforef.be/swi/python.htm> (même livre que pour GRO120)
- Germain, Éric. Guide UML Polytechnique Montréal. Disponible en ligne : https://gigl-uml.github.io/Guide_uml_polymtl/
- Notes de cours en ligne sur les bases de données
 - Concepts et modélisation
 - Guide SQL

6.1. Séquence d'étude suggérée

- Avant la première activité procédurale :
 - Swinnen, chapitre 11 – Classes, objets et attributs (pp. 163-170)
 - Swinnen, chapitre 12 – Classes, méthodes et héritage (pp. 173-187)
 - Guide UML : Diagrammes de classes
 - Notes de cours : Bases de données : Concepts et modélisation
- Avant l'activité en laboratoire :
 - Notes de cours : Base de données : Guide SQL
 - Swinnen, Chapitre 16 – Bases de données (pp. 279-289)
 - Jusqu'à la section " Ébauche d'un logiciel client pour PostgreSQL" exclusivement.
 - Certains éléments déjà couverts dans le guide SQL, mais propose des exemples et une mise en contexte supplémentaires.
- Avant la seconde activité procédurale :
 - Guide UML : Diagrammes de cas d'utilisation, d'interaction et d'états

7. Environnement logiciel

L'ensemble du cours utilisera le langage Python 3 dans l'environnement **uv** et Visual Studio Code en visant les versions disponibles sur les ordinateurs de la faculté. S'il est tout à fait possible d'installer ces logiciels sur vos ordinateurs personnels, et il existe beaucoup de ressources en ligne à cet effet pour vous aider, **sachez que l'équipe d'enseignement de l'APP n'est pas tenue de vous assister dans la configuration de votre équipement personnel**. Si vous éprouvez des difficultés à installer ces logiciels, on vous suggère fortement d'effectuer vos exercices et la résolution de l'APP sur les postes de la faculté.

7.1. Consigne sur l'utilisation de l'IA générative

Dans le cadre de cette unité d'APP, l'utilisation d'outils d'IA est permise pour votre apprentissage personnel (recherche, explication, exemples) et la résolution de la problématique et les livrables associés. Or, l'utilisation d'IA lors des évaluations sommatives et finales n'est évidemment pas permise.

8. Productions à remettre

Vous démontrerez le fonctionnement de votre solution lors de la validation pratique en laboratoire (mardi 30 juin de 13h00 à 16h00). Ensuite, vous devrez rédiger un rapport d'APP (à remettre mercredi le 1^{er} juillet avant 9h00). Les consignes de rédaction du rapport de l'unité d'APP sont données à la section 8.3. Le code de votre solution devra être remis au même moment que le rapport. Les instructions pour le dépôt seront diffusées sur le site de l'APP.

8.1. Formation des équipes

La résolution de la problématique se fera en **équipe de deux**. La problématique peut être résolue entièrement par ordinateur, nous nous attendons à ce que tous les membres de l'équipe soient en mesure d'en faire la démonstration. La façon d'inscrire les équipes sera diffusée sur le site de l'APP. S'il est impossible pour vous de rejoindre une équipe, par exemple car le nombre d'étudiants dans le groupe est impair, veuillez en avvertir le tuteur le plus vite possible.

8.2. Contraintes et pénalités pour les livrables

Veuillez identifier avec votre nom, prénom, numéro de matricule et CIP tous les documents que vous remettrez. Tout retard sur la remise d'un livrable entraîne **une pénalité de 20 % par jour** sur la note associée au rapport de l'APP.

8.3. Livrables

Code

Pour nous permettre d'évaluer votre solution, vous devrez remettre la totalité du code Python et SQL que vous aurez écrit. Celui-ci devra pouvoir s'exécuter sans modifications de l'environnement décrit à la section 7. Le code produit devra être correctement documenté et pourra être exécuté de la façon décrite dans l'énoncé de la problématique. En d'autres termes, la personne qui corrigera votre travail ne devrait pas avoir à modifier quoique ce soit dans votre code pour évaluer son bon fonctionnement.

Rapport

Dans un document de **maximum 5 pages**, vous devrez décrire la façon dont vous avez résolu le problème. Voici les informations recherchées :

Identification du problème. Décrivez clairement le problème à résoudre. Ici, il ne faut pas répéter le texte de l'énoncé de la problématique, mais décrire plus spécifiquement par exemple quel était le problème de normalisation dans le code de démarrage.

Procédure de résolution. Décrivez comment vous avez élaboré la mise en place de la fonctionnalité des bons de travail. **Vous devrez fournir les schémas décrivant le modèle de données complet de l'application (donc vos ajouts et les entités déjà présentes, modifiées ou non), en plus des diagrammes UML décrivant l'ensemble de la solution (états-transitions, séquence, de classes et de cas d'utilisation).**

Mise en œuvre du programme. Cet aspect sera surtout évalué par le code que vous avez produit, mais décrivez ici brièvement la façon dont est organisé votre nouveau code, plus particulièrement où se trouvent les principaux changements.

Analyse du résultat. Présentez une analyse critique du fonctionnement de votre programme. Comment avez-vous validé son fonctionnement ? Quelles sont ses forces ? Quels sont ses limitations et les points à améliorer ? Par exemple, si on voulait ajouter le suivi de robots qui ne sont pas limités à un site, est-ce que le modèle actuel est suffisant ?

9. Évaluations

Cette échelle sera utilisée pour convertir les notes à des cotes (seuil A+ à 90%) :

W ou E	D	D+	C-	C	C+	B-	B	B+	A-	A	A+
<50%	50,0%	54,0%	58,0%	62,0%	66,0%	70,0%	74,0%	78,0%	82,0%	86,0%	90,0%
0,0	1,0	1,3	1,7	2,0	2,3	2,7	3,0	3,3	3,7	4,0	4,3

9.1. Rapport d'APP et validation

La grille suivante décrit la répartition des points pour chaque élément du rapport, du code remis et de la validation :

<i>Activité pédagogique</i>	GRO321	
<i>Compétence</i>	C1	C2
<i>Contenu du rapport</i>		
1. Identification du problème (Q02.1)	5	5
2. Procédure de résolution, incluant diagrammes UML et schéma de base de données (Q02.2)	10	10
3. Mise en œuvre du code (Q02.3)	5	5
4. Analyse du résultat (Q02.4)	5	5
Qualité du code fourni (Q05.2)	5	5
Total du rapport	30	30
Validation de la solution	15	15

C1 : Programmation orientée objet, C2 : Base de données

Les éléments du rapport et de la validation sont évalués selon les indicateurs des qualités de l'ingénieur qui sont détaillés à la section 9.4. **Une version en plus grand format de cette grille est également disponible sur le site de l'APP.**

9.2. Évaluation sommative

L'évaluation sommative porte sur tous les objectifs d'apprentissage de l'unité. C'est un examen à la fois théorique et pratique où **aucune documentation sera permise**.

9.3. Évaluation finale

L'évaluation finale portera également sur tous les objectifs d'apprentissage de l'activité pédagogique GRO321. L'examen aura la même forme que celle de l'évaluation sommative théorique et la documentation ne sera toujours pas permise.

9.4. Grille d'évaluation du rapport et de la validation

Une version PDF est également disponible sur le site de l'APP.

GR0321 - Grille d'évaluation, rapport et validation, été 2026

Élément	Rapport-1 Q02.1	Rapport-2 Q02.2	Rapport-3 Q02.3	Rapport-4 Q02.4	Code Q02.2	Validation-1 Q02.2	Validation-2 Q02.3
Créer un problème de programmation Complément Pondération	5	10	5	5	5	10	5
Niveau	Identifie clairement et de façon exhaustive le problème d'ingénierie : reconnaît sa nature, identifie l'information connue et celle manquante, émet des hypothèses et énonce les résultats escomptés	Élabore une procédure de résolution d'un problème de programmation (code)	Mise en œuvre d'une solution à un problème de programmation (code)	Est capable d'analyser correctement les résultats obtenus et témoigne d'une compréhension fine de leurs limites et de leur portée	Utilise les langages Python et SQL de façon efficace	Applique les méthodes d'investigation appropriées	Analyse du résultat d'une séance d'investigation
5 Excellent	100%	Élabore une procédure ingénieuse de résolution	Applique efficacement et rigoureusement la procédure de résolution choisie	Est capable d'analyser correctement les résultats obtenus et d'en identifier les limites et la portée	Utilise de façon efficace les langages Python et SQL. Démonstre la capacité à optimiser l'utilisation de ces langages.	Applique rigoureusement les méthodes d'investigation appropriées	Analyse correctement et rigoureusement l'ensemble des données recueillies pour en faire ressortir les principales informations utiles
Très bien 4 (cible)	82%	Identifie clairement le problème d'ingénierie : reconnaît sa nature, identifie l'information connue et celle manquante	Applique correctement la procédure de résolution choisie	Est capable d'analyser correctement les résultats obtenus et d'en identifier les limites et la portée	Utilise les langages Python et SQL selon les protocoles établis, sans erreur notable.	Applique les méthodologies d'investigation appropriées	Analyse correctement l'ensemble des données recueillies pour en faire ressortir les principales informations utiles
3 Bien	70%	Identifie convenablement, mais avec quelques lacunes mineures ou imprécisions, le problème d'ingénierie	Élabore une procédure convenant à la résolution du problème, sans être parmi les meilleures	Est capable d'analyser sommairement les résultats obtenus et d'en identifier les limites et la portée	Utilise de façon appropriée les langages Python et SQL, mais présente quelques lacunes mineures	Applique les méthodologies d'investigation appropriées en faisant peu d'erreurs mineures	Analyse correctement certaines données recueillies pour en faire ressortir les principales informations utiles
Passable 2 (seuil)	58%	Perçoit suffisamment, mais avec plusieurs lacunes mineures ou imprécisions, le problème d'ingénierie pour en permettre la résolution	Élabore une procédure de résolution de problème minimalement appropriée	Est capable d'analyser partiellement les résultats obtenus et d'en identifier les limites et la portée	Utilise minimalement les langages Python et SQL	Applique les méthodologies d'investigation appropriées en faisant des erreurs mineures	Analyse partiellement, certaines données recueillies pour en faire ressortir quelques informations utiles
1 Insuffisant	25%	N'arrive pas à cerner le problème d'ingénierie pour permettre sa résolution	Élabore une procédure de résolution de problème incomplète ou inappropriée	Est incapable d'analyser les résultats obtenus et d'en identifier les limites et la portée	N'utilise pas les langages Python et SQL de façon appropriée	N'applique pas les méthodologies d'investigation appropriées ou les applique en faisant des erreurs majeures	N'analyse pas correctement les données recueillies menant à des interprétations erronées
0 Non initié	0%	Ne propose pas d'identification du problème d'ingénierie.	Ne met pas en œuvre une procédure de résolution de problème	N'analyse pas les résultats obtenus	N'utilise pas les langages Python ou SQL	N'applique pas les méthodologies d'investigation appropriées	N'analyse pas les données recueillies

10. Activités procédurale 1

10.1. Programmation orientée objet

Question 1. Quelle est l'utilité de la programmation orientée objet ? Nommez des bénéfices.

Question 2. Identifiez les quatre (4) principes de la programmation orienté objet, puis décrivez chacun des principes.

1. Principe :

- Description :

2. Principe :

- Description :

3. Principe :

- Description :

4. Principe :

- Description :

Question 3. Dans un contexte où on aurait une liste d'animaux pouvant contenir soit des chats ou des chiens et que chacun aurait la méthode « parler » qui lorsqu'appelée, écrirait à la console « miaou » ou « woof » (selon l'objet).

- Quel serait le ou les éléments découlant du principe d'héritage dans le contexte présenté ?
- Quel serait le ou les éléments découlant du principe de polymorphisme ?
- Si la liste d'animaux contient trois objets de type chat et deux de type chien, quelle serait la sortie en console si l'on fait une boucle sur la liste en appelant la méthode « parler » ?

Question 4. Pour cette question, nous allons nous baser sur une classe nommée « **Point** » pouvant contenir une coordonnée « **x** » et « **y** ».

Comment peut-on :

- Définir cette classe en Python ?
- Ajouter une définition à cette classe ?
- Créer un objet « p1 » à partir de cette classe ?
- Afficher la définition associée à cette classe ?
- Ajouter les valeurs 2 et 4 aux attributs « x » et « y » d'un point (sans utiliser le constructeur) ?
- Afficher la valeur associée à l'attribut « x » du point « p1 » ?
- Passer en paramètre l'objet Point à une fonction globale nommée « affiche_point » ?

Question 5. Quel est le rôle du constructeur ? Comment se présente-t-il en Python ? À quel moment est-il appelé ?

Question 6. Comment pourrait-on définir les attributs « x » et « y » de la classe « Point » à l'aide d'un constructeur dont les valeurs par défaut seront zéros (0) ?

Question 7. À partir de la définition de la classe Point, ajouter une nouvelle méthode à cette classe, appelée `afficher_xy()`, qui affiche en console les valeurs de x et y. Puis :

- Créer un objet **p1** ayant la coordonnée (0, 0), puis afficher les valeurs de x et y :
- Créer un objet **p2** ayant la coordonnée (0, 3), puis afficher les valeurs de x et y :
- Créer un objet **p3** ayant la coordonnée (2, 1), puis afficher les valeurs de x et y :

Question 8. Est-ce qu'il existe aussi un *destructeur* en Python ? À quoi pourrait-il servir ?

Exercice de modélisation de classes. Supposons que vous développez une application de dessin vectoriel (pensez au mode *Draft* de Solidworks, en plus primitif). Vous voulez modéliser un ensemble de formes fermées appartenant à l’une des catégories préétablies suivantes : cercle, rectangle, carré, polygone arbitraire.

Voici un exemple des classes qui pourraient être définies pour chacune des catégories :

(Note: le diagramme original, compatible sur draw.io, est disponible sur le site de l'APP)

Rectangle
+ largeur: float64 + hauteur: float64 + centre_x: float64 + centre_y: float64
+ aire(): float64 + déplacer(dx: float64, dy: float64)

Cercle
+ rayon: float64 + centre_x: float64 + centre_y: float64
+ aire(): float64 + déplacer(dx: float64, dy: float64)

Carré
+ taille: float64 + centre_x: float64 + centre_y: float64
+ aire(): float64 + déplacer(dx: float64, dy: float64)

Polygone
+ centre_x: float64 + centre_y: float64 + points_x: float64[3..n] + pointx_y: float64[3..n]
+ aire(): float64 + déplacer(dx: float64, dy: float64)

Question 9. Modifiez cette hiérarchie de classe pour que toutes les formes dérivent d'une classe parente commune et, pour l'instant, sans changer les attributs des classes.

Question 10. Est-ce que certains attributs pourraient être promus à des instances de classes ?
Proposez une modification additionnelle pour isoler la gestion de ces attributs.

10.2. Bases de données

Question 1. Qu'est-ce qu'une base de données ?

Question 2. Qu'est-ce qu'une entité ?

Question 3. Qu'est-ce qu'un attribut ?

Question 4. Qu'est-ce qui distingue un attribut atomique versus un attribut composite ?

- Attribut atomique (ou simple) :
- Attribut composite :

Question 5. Qu'est-ce qu'un identifiant ?

Question 6. Qu'est-ce qu'une clé étrangère ?

Question 7. Qu'est-ce qu'une relation (association) ? Qu'est-ce que la cardinalité d'une relation ?

Question 8. Si vous êtes dans une salle de classe de la Faculté, déterminer 3 entités avec ses attributs, et 3 relations avec leur cardinalité.

Question 9. Si vous aviez à modéliser dans une base de données relationnelle les polygones de l'exercice précédent, quelles entités, attributs et relations vous pourriez proposer ?

Exercice d'algèbre relationnelle. Voici quelques tables basées sur les exemples donnés dans vos notes de cours. Vous pouvez vous référer au schéma de la figure 1 pour comprendre les relations :

Robot

serie	modele	charge_utile
UGV-2041	Husky A300	75
UGV-2042	Husky A300	75
UAV-0317	Vision M650	6
UGV-1107	Jackal C200	20

Mission

code	site
M-042	Aciérie Sorel
M-051	Raffinerie Lévis
M-063	Port de Trois-Rivières

Affectation

serie	code	resultat
UGV-2041	M-042	succes
UGV-2041	M-051	succes
UGV-2042	M-051	echec
UAV-0317	M-063	succes
UGV-2042	M-063	abandon

Question 6. Quel est le résultat de chacune de ces évaluations ?

a) $\sigma \text{ resultat} = \text{'echec'}$ (Affectation)

b) $\pi \text{ modele}$ (Robot)

c) $\pi \text{ serie} (\sigma \text{ charge_utile} \geq 50 \text{ (Robot)})$

d) $\sigma \text{ serie} = \text{'UGV-2042'}$ (Robot $\bowtie$ Affectation)

e) $\pi \text{ site} (\sigma \text{ resultat} = \text{'echec'}$ (Affectation $\bowtie$ Mission))

f) $\pi \text{ serie} (\text{Robot}) - \pi \text{ serie} (\text{Affectation})$

11. Activité en laboratoire

Cette activité fait le pont entre les deux activités procédurales : vous y implémentez la hiérarchie de formes modélisée à la première activité, vous manipulez une base de données SQLite en ligne de commande, puis vous combinez les deux pour rendre vos objets persistants. Les trois exercices sont progressifs et chacun s'appuie sur le précédent.

11.1. Préparation

Récupérez les fichiers de départ : `formes.py` (exercice 1), `formes_schema.sql` et `formes_donnees.sql` (exercice 2), et `formes_bd.py` (exercice 3). Placez-les dans un même dossier. L'environnement du cours est géré par uv : les scripts Python se lancent avec `uv run python fichier.py` et l'interpréteur SQLite avec `uv run sqlite3 lab.db`. On suggère de charger le tout dans Visual Studio Code et interagir avec SQLite depuis le terminal intégré pour l'exercice 2.

11.2. Exercice 1 - Des classes pour les formes

À l'activité procédurale 1, vous avez modélisé une hiérarchie de formes pour une application de dessin vectoriel : une classe parente **Forme** (avec un centre, une méthode `aire()` et une méthode `deplacer()`), et les classes dérivées **Rectangle**, **Cercle**, **Carré** (qui dérive de **Rectangle**) et **Polygone**. Il est temps de passer du diagramme au code. Le fichier `formes.py` contient le squelette : la classe `Point` est fournie comme exemple, et des tests en fin de fichier valident votre travail.

1. Lisez les classes `Point` et `Forme`, puis exécutez le fichier tel quel (`uv run formes.py`). Le test échoue : à quelle ligne, et pourquoi ? Suivez les instructions dans les commentaires de la classe affectée pour régler le problème.
2. Implémentez `deplacer()` dans `Forme`. Toutes les formes en hériteront sans la réécrire.
3. Implémentez le reste de la classe `Cercle` : constructeur et `aire()`.
4. Implémentez `Carré`, qui dérive de `Rectangle`. Son constructeur tient en une ligne. Question : quand vous appelez `k.aire()` et `k.deplacer()` sur un carré, dans quelle classe chaque méthode est-elle réellement définie ?

5. (Défi additionnel) Implémentez Polygone avec la formule du lacet (donnée dans le fichier), et décommentez ses tests. Attention : `deplacer()` doit aussi déplacer les sommets.

Tous les tests doivent afficher « Tous les tests ont réussi! » avant de passer à la suite.

6. (Défi extra) Ajouter le test suivant pour le Polygone (à la fin des tests déjà en place) et valider que tous les tests réussissent.

```
#TODO 6 (Défi extra)
try:
    Polygone(Point(0.0, 0.0), [Point(0.0, 0.0), Point(4.0, 0.0)])
    assert false
except ValueError:
    #Exception détecté!
    pass
```

11.3. Exercice 2 - SQL en ligne de commande

On veut maintenant sauvegarder des dessins. Le schéma fourni comporte deux tables : `dessins` (un dessin a un titre) et `formes` (chaque forme appartient à un dessin, la clé étrangère, avec sa catégorie, son centre et ses dimensions). Ouvrez `formes_schema.sql` et repérez les contraintes : clés primaires, clé étrangère, CHECK. Remarquez aussi que les colonnes `largeur`, `hauteur` et `rayon` ne s'appliquent pas à toutes les catégories, on devine beaucoup de valeur NULL lorsque ces attributs ne s'appliquent pas à la catégorie.

1. **Créer** la base : lancez `uv run sqlite3 lab.db`, puis exécutez `.read formes_schema.sql` et `.read formes_donnees.sql`. Explorez avec `.tables` et `.schema formes`.
2. **Lire**. Affichez toutes les formes (`SELECT * FROM formes;`) et observez les NULL. Écrivez ensuite une requête qui liste les cercles seulement, triés par rayon décroissant.
3. **Créer**. Ajoutez au dessin 1 un cercle de rayon 0.5 centré en (3, 3), en laissant SQLite générer `forme_id`. Vérifiez avec un `SELECT`. Que se passe-t-il si vous tentez une catégorie `'triangle'` ?
4. **Modifier**. Déplacez tous les points du dessin 2 de 5 unités vers la droite, en une seule requête (indice : `SET centre_x = centre_x + 5`). Combien de rangées sont touchées ?

5. **Supprimer.** Supprimez le cercle ajouté à l'étape 3 (par son `forme_id`). Que rapporterait la même requête exécutée une seconde fois ?
6. **Joindre.** Affichez, pour chaque forme, le titre de son dessin et sa catégorie, à l'aide d'un `JOIN` entre les deux tables.

11.4. Exercice 3 - Des objets à la base

Dernière étape : faire communiquer vos classes de l'exercice 1 avec la base de l'exercice 2. Le fichier `formes_bd.py` fournit `get_connection()` et une fonction exemple, `lister_formes()`, qui montre la structure d'une fonction d'accès aux données. Quatre fonctions sont à compléter, et le scénario de test en fin de fichier valide l'ensemble (`uv run python formes_bd.py`).

1. Étudiez `lister_formes()` et exécutez le fichier : la liste des six formes s'affiche, puis le test échoue sur le premier `TODO`.
2. **TODO 1** - `ajouter_forme()` : un `INSERT` dont les colonnes dépendent du type de l'objet reçu. Attention à l'ordre de vos `isinstance` :
 - Pourquoi faut-il tester `Carre` avant `Rectangle` ?
3. **TODO 2** - `lire_forme()` : le chemin inverse. Un `SELECT` dont la rangée sert à reconstruire le bon objet selon la catégorie.
4. **TODO 3 et 4** - `deplacer_forme()` (`UPDATE`) et `supprimer_forme()` (`DELETE`), qui retournent `True` ou `False` selon `cursor.rowcount`.
5. **Réflexion.** Votre base ne peut pas stocker de Polygone. En vous rappelant la discussion de l'activité procédurale (tables Polygone, Point et leur relation), quelles tables faudrait-il ajouter, et pourquoi une simple colonne ne suffit-elle pas ?

11.5. Pour la suite

Deux observations faites aujourd'hui reviendront à la prochaine activité procédurale : la méthode `aire()` de `Forme` qui lève une exception est une version artisanale de ce que les **classes abstraites** formalisent, et les colonnes `NULL` de la table `formes` sont le symptôme d'un schéma à **normaliser**.

12. Activités procédurale 2

12.1. Programmation orientée objet : Classes abstraites et surcharge de fonctions

Une **classe abstraite** définit un concept général dont on ne veut pas créer d'instances directement, mais qui impose un contrat à ses classes dérivées : « toute sous-classe concrète devra fournir ces méthodes ». Une **interface** pousse l'idée à l'extrême : elle ne contient *que* le contrat, sans aucune implémentation. Des langages comme Java ou C# distinguent formellement les deux ; Python, fidèle à sa philosophie, est moins strict : par défaut, rien n'empêche d'instancier une classe « abstraite » ou d'oublier une méthode requise. Le module `abc` (*Abstract Base Classes*) permet de rendre le contrat explicite et vérifié, mais ne fait pas partie du cœur du langage². Pour les interfaces, Python s'appuie traditionnellement sur le ***duck typing***³ : si un objet possède les bonnes méthodes, il « fait l'affaire », sans déclaration formelle. Une classe sans aucune méthode concrète (c'est-à-dire sans implémentation réelle) joue le rôle d'interface explicite lorsqu'on veut imposer le contrat par héritage. Implémenter une méthode dans une classe dérivée de la même signature que celle d'une classe parente s'appelle **la surcharge de méthode**. Supposons cette implémentation simplifiée d'un système de perception pour robot mobile :

```
class Capteur:                                # Classe abstraite
    def lireDerniere(self):                    # Méthode à implémenter
        raise NotImplementedError("lireDerniere()") # Exception soulevée.

class CapteurLidar(Capteur):
    def __init__(self):
        self.data = [1,2,3,4]
    def lireDerniere(self):
        return self.data[-1]                # Retourne la dernière valeur du tableau "data"

class CapteurSonar(Capteur):
    def calculDistRebond(self):
        return 0;                           # Pas encore implémentée

def afficheCapteur(capteur):
    assert(isinstance(capteur, Capteur))      # S'assure qu'on dérive bien de Capteur
    print(capteur.lireDerniere())
```

Question 1. Qu'arrive-t-il si on exécute ce script ?

```
lidar = CapteurLidar()
sonar = CapteurSonar()

afficheCapteur(lidar)
afficheCapteur(sonar)
```

² Voir ici pour de plus amples détails : <https://docs.python.org/3/library/abc.html>

³ Basé sur l'expression anglaise "If it looks like a duck, swims like a duck, and quacks like a duck, it's probably a duck".

12.2. Diagrammes UML

Mise en situation. La pizzeria Chez Mario lance son système de commande en ligne. Un **client** y passe sa commande, peut en suivre la progression et peut l'annuler tant que la préparation n'a pas commencé. En cuisine, le **pizzaiolo** signale au système qu'une commande est prête ; le **livreur** la prend alors en charge et confirme la livraison une fois la pizza remise au client.

Chaque commande suit le cycle de vie suivant : elle est d'abord **reçue** ; une fois le paiement confirmé, elle passe **en préparation** ; quand le pizzaiolo la signale, elle devient **prête** ; à sa prise en charge par le livreur, elle passe **en livraison** ; à la remise au client, elle est **livrée**. Un client peut annuler une commande reçue (elle devient **annulée**), et une livraison qui échoue (client introuvable) ramène la commande à l'état prête, en attente d'une nouvelle tentative.

Lorsqu'un client passe une commande, le site web transmet la demande au gestionnaire de commandes, qui vérifie dans la base de données que l'adresse est dans la zone de livraison avant d'enregistrer la commande et de retourner une confirmation avec le délai estimé.

Question 1 - Cas d'utilisation. Dessinez le diagramme de cas d'utilisation du système : les acteurs, au moins quatre cas d'utilisation, et au moins une relation « *include* » ou « *extend* ».

Question 2 - Séquence. Dessinez le diagramme de séquence du scénario « passer une commande en ligne », en faisant intervenir le client, le site web, le gestionnaire de commandes et la base de données. Distinguez les appels des retours.

Question 3 - États-transitions. Dessinez le diagramme d'états-transitions du cycle de vie d'une commande, avec les événements qui déclenchent chaque transition, un état initial et les états finaux.

Question 4 - Réflexion. On ajoute la règle suivante : « le client peut aussi annuler une commande déjà en préparation, moyennant des frais ». Lequel de vos diagrammes faut-il modifier, et comment ?

12.3. Base de données : Normalisation

Question 1 – Quel est le but de la normalisation ?

Question 2 – Énumérer quelques problèmes potentiels inhérent à l'utilisation d'une table non normalisée.

Reprenons la mise en situation du problème précédent. Pour son système de commande en ligne, la pizzeria Chez Mario a confié un premier prototype à un développeur pressé, qui a entassé toutes les commandes dans une seule table, *Commande*, dont la clé primaire est *no_commande* :

no_commande	date	telephone	nom_client	adresse	pizzas
C-101	2026-06-01	819-555-1234	Alice Roy	12, rue des Pins	1 Margherita à 12 \$; 2 Végé à 14 \$
C-102	2026-06-01	819-555-8888	Benoit Lao	34, av. du Parc	1 Quatre fromages à 16 \$
C-103	2026-06-02	819-555-1234	Alice Roy	12, rue des Pins	1 Végé à 14 \$
C-104	2026-06-03	819-555-2345	Carmen Diaz	56, boul. Iberville	2 Margherita à 12 \$; 1 Quatre fromages à 16 \$

Le téléphone identifie le client (deux clients ne partagent jamais un numéro), et chaque pizza du menu a un nom unique et un prix fixe. Vous devez normaliser ce schéma, une forme normale à la fois.

Question 3 - Première forme normale. Expliquez pourquoi la table ne respecte pas la 1FN, puis proposez une décomposition qui la respecte (indice : une rangée par pizza commandée). Donnez la clé primaire de chaque table obtenue.

Question 4 - Deuxième forme normale. Dans votre table des lignes de commande (*no_commande*, *pizza*, *quantité*, *prix unitaire*), identifiez la dépendance fonctionnelle qui viole la 2FN et décrivez une anomalie concrète qu'elle cause. Corrigez le schéma.

Question 5 - Troisième forme normale. Montrez que la table des commandes restante viole la 3FN en exhibant une dépendance transitive, et décrivez une anomalie concrète visible dans les données ci-dessus. Corrigez le schéma.

Question 6 - Schéma final. Donnez le schéma complet obtenu, en indiquant la clé primaire de chaque relation et les clés étrangères.

13. Validation pratique de la solution à la problématique

L'ordre de présentation des équipes sera aléatoire et sera diffusé sur le site de l'APP. Lors de la validation, vous aurez à démontrer le bon fonctionnement de votre solution et expliquer comment vous avez résolu le problème. Référez-vous à la grille de la section 9.4 (dernières colonnes) pour connaître les critères d'évaluation. La personne évaluant votre travail pourrait vous poser des questions individuellement à chaque membre de l'équipe – assurez-vous que chaque membre soit en mesure de décrire l'ensemble de la solution.

14. Deuxième tutorat

Le deuxième tutorat est l'occasion pour vous de partager vos solutions à la problématique et confirmer vos acquis. Votre tuteur ou tutrice vous guidera dans la discussion. Pensez à prendre des notes, par exemple en dessinant un schéma relationnel des concepts abordés. Pensez à relire la liste des connaissances à acquérir à la section 5 et n'hésitez surtout pas à poser des questions si certains de ces concepts ne sont pas encore clairs pour vous.